

An Archetypal Analysis of ICML (2019) Conference

732A76 Research Project

Thalgamuwe Gedara Ashen Akalanka Weligalle

December 17, 2025

Abstract

Archetypal Analysis (AA) was introduced by Cutler and Breiman in 1994 [1]. It is a method designed to extract distinct structural patterns, called archetypes, from observational data, where each data point is approximated as a convex mixture of these archetypes [2]. Although AA has been successfully applied in several domains, its potential as a topic modeling framework for scientific literature remains relatively underexplored. In this project, we investigate the use of Archetypal Analysis as a topic modeling approach for analyzing research abstracts related to ICML conference-2019 6.3.1. thereby creating a self-referential analysis of the literature.

The experimental results demonstrate that Archetypal Analysis (AA) produces highly interpretable and sparse topic representations that effectively capture extremal semantic structures within the literature, often more distinctly than probabilistic or centroid-based methods. This work highlights the suitability of Archetypal Analysis as a geometric framework for topic modeling and provides insights into the structure and evolution of research trends within the AA research community. The implementation and experimental results are publicly available at https://github.com/AshenWELI/Archetypal_Analysis.

1 Introduction

Archetypal Analysis (AA), introduced by Cutler and Breiman [1], provides the theoretical foundation for this project. According to their work, the original motivation for Archetypal Analysis arose from a practical problem in 1988, when the Swiss Army needed to design improved protective face masks for soldiers. This task was assigned to Flury and Riedwyl, who analyzed a large dataset of human head measurements and observed interesting geometric patterns in the data.[1]

A natural question that emerged from this study was whether a small number of “pure” head types, called *archetypes*, could explain the observed variability in the dataset. In other words, could the roughly 200 measured head shapes be represented as convex mixtures of a few extreme, idealized head shapes? To address this problem, Cutler and Breiman developed Archetypal Analysis.

In this work, we build on the theoretical framework of Archetypal Analysis and explore its application as a topic modeling method for scientific text data.

1.1 Archetypal Analysis

Let’s consider a dataset represented by a matrix $\mathbf{X} \in \mathbb{R}^{N \times M}$, where N is the number of data samples and M is the number of features. The dataset can be written as

$$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\},$$

where each $\mathbf{x}_i \in \mathbb{R}^M$ represents one observation.

The goal of Archetypal Analysis is to approximate the data matrix $\mathbf{X}$ by a low-rank decomposition

$$\mathbf{X} \approx \mathbf{S}\mathbf{A},$$

where $\mathbf{S} \in \mathbb{R}^{N \times K}$ and $\mathbf{A} \in \mathbb{R}^{K \times M}$.

Here, $\mathbf{S}$ is the coefficient (or weight) matrix that expresses each data point as a convex combination of archetypes, and $\mathbf{A}$ is the matrix containing the K archetypes themselves. [2] The common data

representation in Archetypal Analysis expresses each data point x_i as a convex combination of K archetypes $a_1, a_2, \dots, a_K$:

$$x_i \approx A^\top s_i,$$

where $s_i \geq 0$ and $\sum_{j=1}^K s_{ij} = 1$. The vector s_i contains the weights of the convex combination, and these vectors are stacked row-wise to form the matrix $S \in \mathbb{R}^{N \times K}$. The matrix $A \in \mathbb{R}^{K \times M}$ contains the archetypes $a_1, a_2, \dots, a_K$ as its rows and is referred to as the design matrix of archetypes.

Each archetype can itself be expressed as a convex combination of the original data points:

$$a_k = X^\top c_k,$$

where $c_k \geq 0$ and $\sum_{i=1}^N c_{ki} = 1$. These coefficient vectors are stacked row-wise to form the matrix

$$C \in \mathbb{R}^{K \times N}.$$

From these definitions, it follows that

$$A = CX.$$

Due to the convexity constraints, both coefficient matrices S and C are row-stochastic, meaning that all entries are non-negative and each row sums to one.

1.2 Residual Sum of Squares (RSS)

The weight matrices are obtained by minimizing the Residual Sum of Squares (RSS) as the objective function, defined as

$$\text{RSS} = \|X - SCX\|_F^2 = \|X - SA\|_F^2.$$

Here, $\|\cdot\|_F^2$ denotes the squared Frobenius norm.

Optimizing the RSS in order to estimate the optimal weight matrices S and C leads to a non-convex optimization problem. However, the problem is convex in S when C is fixed, and convex in C when S is fixed [2].

For readers interested in the optimization procedure, the pseudocode is presented in Subsection 6.1.1. Further technical details can be found in the references [2], [3].

1.3 Geometric Interpretation

The minimization of the residual sum of squares (RSS) in Archetypal Analysis can be interpreted from a geometric perspective. In this view, the archetypes are selected as extremal points that span a large convex hull (illustrated as the light-blue region), while each data point is approximated by its projection onto this convex hull. [2] The grey line segments represent the shortest Euclidean distances between each observation and its corresponding projection, and the colored points indicate the dominant archetype to which each observation is most strongly associated.

Formally, this geometric interpretation expresses the RSS as the sum of squared distances between each data point and the convex hull of the archetypes [1], [3]:

$$\text{RSS} = \sum_{n=1}^N \min_{x' \in \text{conv}(\{a_1, \dots, a_K\})} \|x_n - x'\|_2^2,$$

where $\text{conv}(\cdot)$ denotes the convex hull and $a_1, \dots, a_K$ are the archetypes.

Beyond Euclidean distance, this geometric formulation naturally admits an angular interpretation. Since archetypes lie on the boundary of the convex hull and correspond to extremal directions in the data space, the cosine of the angle between a data point x_n and an archetype a_k provides a measure of directional alignment. A small cosine angle (i.e., high cosine similarity) indicates that the data point lies close to the ray defined by the archetype, even if its magnitude differs. Consequently, cosine similarity

serves as a complementary interpretability measure that highlights semantic alignment with archetypal directions, particularly in high-dimensional text representations where vector norms vary substantially.

This directional viewpoint reinforces the interpretation of archetypes as extreme semantic patterns rather than centroids, distinguishing Archetypal Analysis from clustering methods such as k -means and aligning it more closely with geometric topic modeling approaches [1], [3].

1.4 Research Aim and Targets

The primary aim of this project is to investigate Archetypal Analysis (AA) as a topic modeling framework for textual data, with a particular focus on scientific abstracts from the ICML 2019 conference. By applying AA to research literature related to Archetypal Analysis itself, this study seeks to uncover the underlying semantic structure and thematic organization of the corpus.

Specifically, this project aims to:

- Evaluate the suitability of Archetypal Analysis for modeling high-dimensional categorical text data.
- Reproduce and validate previous findings on text-based Archetypal Analysis reported in [3].
- Compare the performance and interpretability of AA against established topic modeling methods, including Non-Negative Matrix Factorization (NMF) 6.2.1, k -means clustering 6.2.2 [4] [5], and Latent Dirichlet Allocation (LDA) [6].
- Analyze the geometric and sparsity characteristics of AA-derived topic representations.

2 Methodology

This section describes the methodological framework used in this research, outlining the steps taken to design, validate, and apply the Archetypal Analysis (AA) model for topic extraction from scientific abstracts. The methodology follows a structured pipeline to ensure that the analysis is both technically sound and empirically validated.

The first stage of the project involves implementing an AA model from scratch. Since AA is sensitive to initialization and non-convex optimization, it is essential to verify the correctness of the implementation. The algorithmic foundations are based on the description provided in Alcacer et al. [2], particularly Section II, with additional inspiration from the implementation details in Mørup and Hansen [3]. To ensure reliability, the custom AA implementation is validated using the NIPS Bag-of-Words dataset (available at the UCI Machine Learning Repository), following the evaluation setup used in [3].

Once the AA implementation is validated, the second stage focuses on applying the model to the collected corpus of ICLR 2019 abstracts. The extracted topics (archetypes) are then compared against those produced by established topic modeling methods, including K-means clustering [4], [5], Non-Negative Matrix Factorization (NMF; see Section 6.2.1), and Latent Dirichlet Allocation (LDA)[6]. This comparative analysis allows us to assess differences in interpretability, sparsity, topic coherence, and reconstruction error across methods.

Together, these steps form a comprehensive methodology that ensures both the correctness of the AA implementation and the validity of its use as a topic modeling framework.

3 Experimental Results

This chapter presents the experimental evaluation of the proposed methodology. The experiments assess the effectiveness of Archetypal Analysis (AA) as a topic modeling approach for textual data and compare its performance with established baseline methods: Non-Negative Matrix Factorization (NMF) 6.2.1, k -means clustering [5] 6.2.2 [4], and Latent Dirichlet Allocation (LDA) [6]. The evaluation includes both validation experiments and large-scale experiments conducted on conference data.

3.1 Data

Two publicly available text datasets are used in this study. The first dataset is employed for model validation and parameter calibration, while the second dataset consists of conference abstracts used for the main experimental analysis. Both datasets are of substantial size and enable reproducibility.

For validation experiments, the number of documents is restricted to a maximum of 500 to control computational cost while preserving sufficient semantic structure. The number of topics (archetypes) is fixed to $K = 10$ across all models to ensure a fair comparison.

3.2 Model Validation with Test Data

Table 1 reports quantitative results obtained from the validation dataset. The evaluation metrics include reconstruction error, computational time, sparsity-related measures, and method-specific quality indicators.

Table 1: Summary statistics for topic modeling methods on validation data ($K = 10$, max_docs=500).

Metric	AA	NMF	k -means	LDA
Reconstruction Error (RSS)	3.31×10^2	3.17×10^2	3.47×10^2	2.27×10^7
RSS per Document	6.62×10^{-1}	6.33×10^{-1}	6.95×10^{-1}	4.53×10^4
Computation Time (s)	2217.36	4.85	2.42	3.51
Avg. Active Components / Document	4.16	6.21	1.00	1.53
Silhouette Score (k -means)	–	–	0.021	–
Calinski–Harabasz Index	–	–	7.65	–
Perplexity (LDA)	–	–	–	18863.92

3.2.1 Geometric and Interpretability Analysis

Additional geometric and sparsity-related metrics are summarized in Table 2. These measures highlight structural differences in how each method represents documents in topic space.

Table 2: Comparative analysis of topic modeling methods on validation data.

Metric	NMF	AA	k -means	LDA
Non-zero entries in S (%)	66.5	43.5	10.0	100.0
Avg. cosine angle (degrees)	81.0	64.9	51.1	18.9
Reconstruction error (Frobenius)	68.16	18.19	18.64	4760.70
Avg. active components / document	6.65	4.35	1.00	10.00

3.3 Experimental Results on Conference Data

This subsection presents results obtained from the main experimental run on the conference abstract dataset using $K = 10$ topics. All methods are applied to the same normalized document–term matrix to ensure comparability.

3.3.1 Summary Statistics

Table 3 reports the primary quantitative metrics, capturing reconstruction quality, computational efficiency, and component usage.

Table 3: Summary statistics for topic modeling methods on conference data ($K = 10$).

Metric	AA	NMF	k -means	LDA
Reconstruction Error (RSS)	4.50×10^2	4.46×10^2	4.57×10^2	2.62×10^6
RSS per Document	8.99×10^{-1}	8.92×10^{-1}	9.15×10^{-1}	5.23×10^3
Computation Time (s)	524.52	1.47	1.81	2.72
Avg. Active Components / Document	4.28	4.79	1.00	9.83
Silhouette Score (k -means)	–	–	0.008	–
Calinski–Harabasz Index	–	–	2.58	–
Perplexity (LDA)	–	–	–	27328.50

3.3.2 Geometric Comparison

Table 4 summarizes sparsity, cosine-angle separation, reconstruction accuracy, and topic usage for the conference dataset.

Table 4: Geometric and sparsity comparison on conference data.

Metric	NMF	AA	k -means	LDA
Non-zero entries in S (%)	51.7	46.6	10.0	100.0
Avg. cosine angle (degrees)	79.1	73.8	64.1	46.4
Reconstruction error (Frobenius)	41.45	21.20	21.39	1617.40
Avg. active components / document	5.17	4.66	1.00	10.00

4 Discussion

Across both datasets, Archetypal Analysis consistently produces sparse and interpretable document representations while maintaining competitive reconstruction accuracy. Compared to NMF and LDA, AA activates fewer components per document and exhibits larger cosine-angle separation, indicating more distinct semantic directions. These results support the suitability of AA as a geometric topic modeling framework that complements probabilistic and centroid-based approaches for scientific text analysis.

* explain top word list and rss. * success or failier- conclusion * mention references everywhere * compare numbers

cosine angle-

AA has larger average cosine angles than LDA

Meaning:

Topics are more distinct Lower $\rightarrow$ more sparse

5 Conclusion

References

- [1] A. Cutler and L. Breiman, “Archetypal analysis,” *Technometrics*, vol. 36, no. 4, pp. 338–347, 1994.
- [2] A. Alcacer, I. Epifanio, S. Mair, and M. Mørup, “A survey on archetypal analysis,” *arXiv preprint arXiv:2504.12392*, 2025.
- [3] M. Mørup and L. K. Hansen, “Archetypal analysis for machine learning and data mining,” *Neuro-computing*, vol. 80, pp. 54–63, 2012.

- [4] A. M. Ikotun, A. E. Ezugwu, L. Abualigah, B. Abuhaija, and J. Heming, "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data," *Information Sciences*, vol. 622, pp. 178–210, 2023.
- [5] L. Jing, M. K. Ng, X. Yang, and J. Z. Huang, "A text clustering system based on k-means type subspace clustering and ontology," *International Journal of Intelligent Technology*, vol. 1, no. 2, pp. 91–103, 2006.
- [6] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *Journal of machine Learning research*, vol. 3, no. Jan, pp. 993–1022, 2003.
- [7] D. Lee and H. S. Seung, "Algorithms for non-negative matrix factorization," *Advances in neural information processing systems*, vol. 13, 2000.
- [8] M. W. Berry, N. Gillis, and F. Glineur, "Document classification using nonnegative matrix factorization and underapproximation," in *2009 IEEE International Symposium on Circuits and Systems*, IEEE, 2009, pp. 2782–2785.
- [9] A. A. Shetkar and S. Fernandes, "Text categorization of documents using k-means and k-means++ clustering algorithm," *Int J Recent Innov Tren Comput Commun*, vol. 4, no. 6, pp. 485–489, 2016.

6 Appendix

6.1 Code Chunks

6.1.1 AA model

```
import numpy as np
```

```
def project_to_simplex(v):
    """
    Project a vector v onto the probability simplex:
    {w | w >= 0, sum(w) = 1}.
    """
    v = np.asarray(v, dtype=float)
    n = v.shape[0]
    u = np.sort(v)[::-1]
    cssv = np.cumsum(u)
    rho = np.nonzero(u * np.arange(1, n + 1) > (cssv - 1))[0][-1]
    theta = (cssv[rho] - 1.0) / (rho + 1.0)
    w = np.maximum(v - theta, 0.0)
    return w

def solve_simplex_least_squares(B, y, max_iter=500, tol=1e-6):
    """
    Solve: min_s || B^T s - y ||^2 subject to s >= 0, sum(s) = 1
    using projected gradient descent (PGD).

    Note: B should be passed as the matrix (not transposed), and this
    function will compute B^T @ (B^T @ s - y) for the gradient.
    """
    K = B.shape[0] # B is (M, K) so we want K archetypes
    s = np.ones(K) / K # start uniform

    # For min ||B^T s - y||^2, gradient is: B @ (B^T @ s - y)
    # Step size based on Lipschitz constant
    step_size = 1.0 / (np.linalg.norm(B, 2) ** 2 + 1e-12)

    prev_val = np.inf
    for _ in range(max_iter):
        grad = B @ (B.T @ s - y) # Gradient for ||B^T s - y||^2
        s = s - step_size * grad
        s = project_to_simplex(s)

        val = 0.5 * np.linalg.norm(B.T @ s - y) ** 2
        if abs(prev_val - val) < tol * (1 + prev_val):
            break
        prev_val = val
    return s

def archetypal_analysis(X, K, max_iter=50, tol=1e-5, random_state=0,
    verbose=True):
    """
    Archetypal Analysis (AA) via alternating optimization.
```

Follows Algorithm 1: $\min ||X - S^*C^*X||_F^2 = \min ||X - S^*A||_F^2$
where $A = C^*X$ are the archetypes.

Args:

X: (N, M) data matrix (N samples, M features)
K: number of archetypes
max_iter: maximum iterations
tol: convergence tolerance
random_state: random seed

Returns:

S: (N, K) coefficients for data points in archetype space ($s_n \geq 0$, $\sum=1$)
C: (K, N) coefficients for archetypes in data space ($c_k \geq 0$, $\sum=1$)
A: (K, M) archetypes where $A = C^*X$

"""

rng = np.random.default_rng(random_state)
N, M = X.shape

--- Initialize A with random rows of X ---
idx = rng.choice(N, size=K, replace=False)
A = X[idx, :].copy()

S = np.ones((N, K)) / K
C = np.ones((K, N)) / N
prev_obj = np.inf

for iteration in range(max_iter):
--- Step 1: Update S row by row ---
For each data point n, solve: $\min ||A^T s_n - x_n||^2$ s.t. $s_n \geq 0$, $\sum(s_n) = 1$
for n in range(N):
S[n, :] = solve_simplex_least_squares(A, X[n, :])

--- Step 2: Update A via least squares ---
$A = (S^T S)^{-1} S^T X$
StS = S.T @ S + 1e-10 * np.eye(K) # Add small regularization for stability
A = np.linalg.solve(StS, S.T @ X)

--- Step 3: Update C row by row ---
For each archetype k, solve: $\min ||X^T c_k - a_k||^2$ s.t. $c_k \geq 0$, $\sum(c_k) = 1$
for k in range(K):
C[k, :] = solve_simplex_least_squares(X, A[k, :])

--- Step 4: Update A using C ---
$A = C * X$ (archetypes are convex combinations of data points)
A = C @ X

--- Check convergence ---
$RSS = ||X - S^*A||_F^2 = ||X - S^*C^*X||_F^2$
obj = 0.5 * np.linalg.norm(X - S @ A, "fro") ** 2

if verbose and iteration % 5 == 0:
print(f"Iteration {iteration:3d}, RSS = {obj:.4e}")
if abs(prev_obj - obj) < tol * (1 + prev_obj):


```

        if verbose:
            print(f"Converged at iteration {iteration}, RSS = {obj:.4e}
                  ")
        break
    prev_obj = obj

    return S, C, A

```

6.1.2 NMF model

```

from sklearn.decomposition import NMF

def run_nmf(X, K, random_state=0, verbose=False):
    """Run Non-negative Matrix Factorization."""
    if verbose:
        print(f"\nRunning NMF with K={K}...")

    start_time = time.time()
    model = NMF(n_components=K, init='random', random_state=random_state,
               max_iter=200, verbose=0)
    W = model.fit_transform(X)  # Document-topic matrix (N x K)
    H = model.components_      # Topic-word matrix (K x M)

    # Normalize W to get probability distribution
    W_norm = W / (W.sum(axis=1, keepdims=True) + 1e-10)

    elapsed = time.time() - start_time

    # Compute reconstruction error
    reconstruction = W @ H
    rss = np.linalg.norm(X - reconstruction, 'fro') ** 2

    if verbose:
        print(f"  Completed in {elapsed:.2f}s, RSS = {rss:.4e}")

    return W_norm, H, rss, elapsed

```

6.1.3 K-means

```

from sklearn.cluster import KMeans

def run_kmeans(X, K, random_state=0, verbose=False):
    """Run K-means clustering."""
    if verbose:
        print(f"\nRunning K-means with K={K}...")

    start_time = time.time()
    model = KMeans(n_clusters=K, random_state=random_state, n_init=10,
                  max_iter=300)
    labels = model.fit_predict(X)
    centers = model.cluster_centers_  # (K x M)

    # Create one-hot encoding for hard assignments
    W = np.zeros((X.shape[0], K))
    W[np.arange(X.shape[0]), labels] = 1.0

```

```

elapsed = time.time() - start_time

# Compute reconstruction error
reconstruction = W @ centers
rss = np.linalg.norm(X - reconstruction, 'fro') ** 2

# Compute clustering metrics
silhouette = silhouette_score(X, labels)
calinski = calinski_harabasz_score(X, labels)

if verbose:
    print(f" Completed in {elapsed:.2f}s, RSS = {rss:.4e}")
    print(f" Silhouette Score: {silhouette:.4f}, Calinski-Harabasz: {
        calinski:.2f}")

return W, centers, rss, elapsed, silhouette, calinski

```

6.1.4 LDA

```

from sklearn.decomposition import LatentDirichletAllocation

def run_lda(X, K, random_state=0, verbose=False):
    """Run Latent Dirichlet Allocation."""
    if verbose:
        print(f"\nRunning LDA with K={K}...")

    start_time = time.time()
    model = LatentDirichletAllocation(n_components=K, random_state=
        random_state,
                                    max_iter=20, learning_method='batch',
                                    verbose=0)
    W = model.fit_transform(X) # Document-topic distribution (N x K)
    H = model.components_     # Topic-word distribution (K x M)

    elapsed = time.time() - start_time

    # Compute reconstruction error (approximate for LDA)
    reconstruction = W @ H
    rss = np.linalg.norm(X - reconstruction, 'fro') ** 2

    # Compute perplexity
    perplexity = model.perplexity(X)

    if verbose:
        print(f" Completed in {elapsed:.2f}s, RSS = {rss:.4e}")
        print(f" Perplexity: {perplexity:.2f}")

    return W, H, rss, elapsed, perplexity

```

6.2 Method Explanations

6.2.1 Non-Negative Matrix Factorization (NMF)

Unsupervised learning algorithms such as Principal Component Analysis (PCA) and vector quantization can be viewed as matrix factorization methods with different constraints. Non-Negative Matrix Factorization (NMF), introduced by Lee and Seung [7], factorizes a non-negative data matrix into two lower-dimensional non-negative matrices:

$$\mathbf{X} \approx \mathbf{W}\mathbf{H}.$$

Here, $\mathbf{X} \in \mathbb{R}_+^{N \times M}$ denotes the document–term matrix, where each entry represents the frequency or TF-IDF weight of a word appearing in a document. The matrix $\mathbf{W} \in \mathbb{R}_+^{N \times K}$ is the document–topic matrix, where each row indicates how strongly each of the K topics contributes to a given document. The matrix $\mathbf{H} \in \mathbb{R}_+^{K \times M}$ is the topic–word matrix, in which each row represents a topic described by its most important words.

Because all values are non-negative, NMF provides an additive, parts-based representation that is intuitive for topic modeling. Through factorization, NMF attempts to uncover hidden structure in $\mathbf{X}$ by expressing each document as a mixture of a small number of coherent topics [8].

The NMF objective function aims to find matrices $\mathbf{W}$ and $\mathbf{H}$ that minimize the reconstruction error measured by the Frobenius norm:

$$f(\mathbf{W}, \mathbf{H}) = \min_{\mathbf{W}, \mathbf{H} \geq 0} \|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2.$$

Lee and Seung [7] proposed multiplicative update rules that guarantee the objective function is non-increasing under each update. The updates for the Euclidean distance minimization are:

$$H_{a\mu} \leftarrow H_{a\mu} \frac{(\mathbf{W}^\top \mathbf{X})_{a\mu}}{(\mathbf{W}^\top \mathbf{W}\mathbf{H})_{a\mu}},$$

$$W_{ia} \leftarrow W_{ia} \frac{(\mathbf{X}\mathbf{H}^\top)_{ia}}{(\mathbf{W}\mathbf{H}\mathbf{H}^\top)_{ia}}.$$

The reconstruction error remains unchanged under these updates if and only if $\mathbf{W}$ and $\mathbf{H}$ are at a stationary point of the objective function.

It should be noted that, in this project, NMF is not implemented from scratch. Instead, the experiments employ the optimized NMF implementation provided by `scikit-learn`, as described in Section 6.1.2.

6.2.2 K-means

In text clustering, documents are represented as high-dimensional vectors using term frequency–inverse document frequency (TF–IDF) weighting. TF–IDF quantifies the importance of a word within a document relative to the entire corpus by combining two components: term frequency (TF) and inverse document frequency (IDF) [9].

For a given word w in document d , the TF–IDF weight is defined as

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \text{IDF}(w),$$

where

$$\text{TF}(w, d) = \frac{K}{T},$$

with K denoting the number of occurrences of word w in document d , and T the total number of words in document d . The inverse document frequency is given by

$$\text{IDF}(w) = \log \left(\frac{D}{\text{DF}(w)} \right),$$

where D is the total number of documents in the corpus and $\text{DF}(w)$ is the number of documents containing the word w .

After vectorization, the k -means algorithm partitions the document set into K clusters by minimizing the within-cluster sum of squared distances,

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2,$$

where C_k denotes the k -th cluster and μ_k is the centroid, computed as the mean of all document vectors assigned to that cluster. [4]

For text data, cosine similarity is commonly used instead of Euclidean distance in order to reduce sensitivity to document length and sparsity. Under this formulation, clustering is based on angular similarity between document vectors rather than their magnitude. [4]

In this study, k -means serves as a baseline clustering method. Due to its hard assignment mechanism, each document is associated with exactly one cluster, resulting in binary cluster memberships (0 or 1). While this limits interpretability and sparsity compared to Archetypal Analysis, k -means remains useful for identifying coarse-grained groupings of documents and dominant word patterns within clusters.

6.3 Results and Data

This section presents the experimental results, supporting data, and visualizations obtained during the study. It provides additional quantitative and qualitative insights that complement the methodological description given in the previous sections. The figures included in this section illustrate the behavior of the proposed models and highlight key patterns observed in the data.

All visualizations and plots were generated using the experimental code developed as part of this research and are intended to support the interpretation of the results.

6.3.1 Data

This research uses two types of datasets: a benchmark bag-of-words corpus and a collection of scientific conference abstracts. The benchmark dataset is used to validate and calibrate the Archetypal Analysis (AA) implementation developed from scratch, while the conference abstract dataset is used for the main experimental analysis.

Bag-of-Words Dataset The benchmark dataset is obtained from the UCI Machine Learning Repository¹. In this study, the NIPS subset of the Bag-of-Words corpus is used. The dataset consists of $N = 1500$ documents and a vocabulary of $M = 12,419$ unique words, with approximately 6.4 million word occurrences in total. This dataset is commonly used in the literature and provides a reliable baseline for validating topic modeling methods.

Conference Abstract Dataset The second dataset consists of research paper abstracts collected from major machine learning conferences using the OpenReview API (<https://api.openreview.net>). A custom data collection script was developed to automatically download abstracts from conferences such as ICLR, NeurIPS, and ICML. This design allows flexibility in selecting different conferences and publication years for analysis.

The following commands illustrate how the abstract downloader can be used to retrieve different datasets:

```
# Smaller, manageable dataset (recommended for initial experiments)
python abstract_downloader.py --conference ICLR --year 2019

# Larger and more recent dataset
python abstract_downloader.py --conference ICLR --year 2023

# Cross-conference comparison for the same year
python abstract_downloader.py --conference ICLR --year 2019
python abstract_downloader.py --conference NeurIPS --year 2019
python abstract_downloader.py --conference ICML --year 2019
```

¹<http://archive.ics.uci.edu/ml/datasets/Bag+of+Words>

For the experiments reported in this work, the ICLR 2019 dataset was primarily used. Summary statistics for this dataset are shown below:

Conference: ICLR 2019
Total papers: 1419
Average abstract length: 1120 characters
Papers with keywords: 1315 (92.7%)

6.3.2 Results

In this study, Archetypal Analysis (AA) was evaluated under multiple experimental configurations in order to assess its geometric behavior and robustness across datasets. Experiments were conducted on two datasets: a Bag-of-Words (BoW) benchmark corpus and a collection of ICML 2019 conference abstracts. For both datasets, the number of documents was varied ($\text{max_docs} = 500$ and 1000), and the number of archetypes was set to $K \in \{4, 5, 6, 10\}$.

We first present Convex hull obtained from the Bag-of-Words dataset, which was used primarily for model validation. In this setting, clear convex hull structures are observed for $K = 4$, $K = 5$, and $K = 10$, indicating that the learned archetypes correspond to extreme points of the data distribution. This behavior is consistent with the theoretical foundations of Archetypal Analysis.

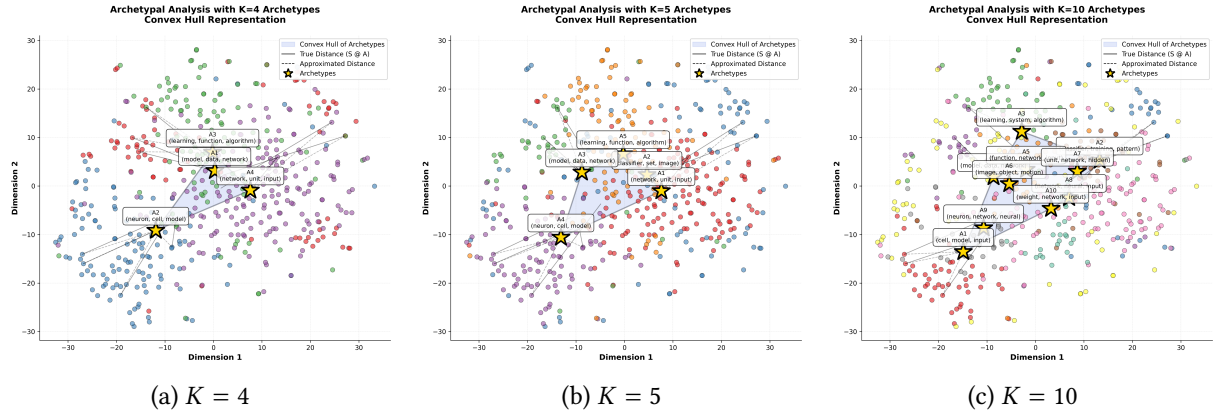


Figure 1: Two-dimensional visualization of Archetypal Analysis for the Bag-of-Words dataset with varying numbers of archetypes ($K = 4, 5, 10$). The archetypes $\{a_1, \dots, a_K\}$ define the vertices of the convex hull, while data points are represented as convex combinations of these extremes.

We next consider the main experimental dataset consisting of ICML 2019 conference abstracts. The same experimental conditions were applied as in the validation experiments. As shown in Figure 2, the convex hull structure persists across different values of K , demonstrating that Archetypal Analysis consistently identifies extremal semantic patterns in real-world scientific text data.

Overall, these visualizations confirm that Archetypal Analysis successfully captures the geometric structure of both benchmark and real-world text datasets. The persistence of convex hull formations across datasets and values of K provides empirical support for AA as a robust geometric topic modeling framework.

We now analyze the extracted word distributions for the case $K = 10$. Figure 3 presents the top words associated with each of the ten components obtained using Non-Negative Matrix Factorization (NMF), Archetypal Analysis (AA), k -means clustering, and Latent Dirichlet Allocation (LDA) on the Bag-of-Words dataset. The components are ordered by importance, and the grayscale intensity indicates the relative contribution of each word within a component.

We now turn to the experimental dataset consisting of ICML 2019 conference abstracts. Figure 4 shows the corresponding word distributions for the $K = 10$ component models. The extracted components reveal clearly distinguishable thematic structures within the ICML corpus.

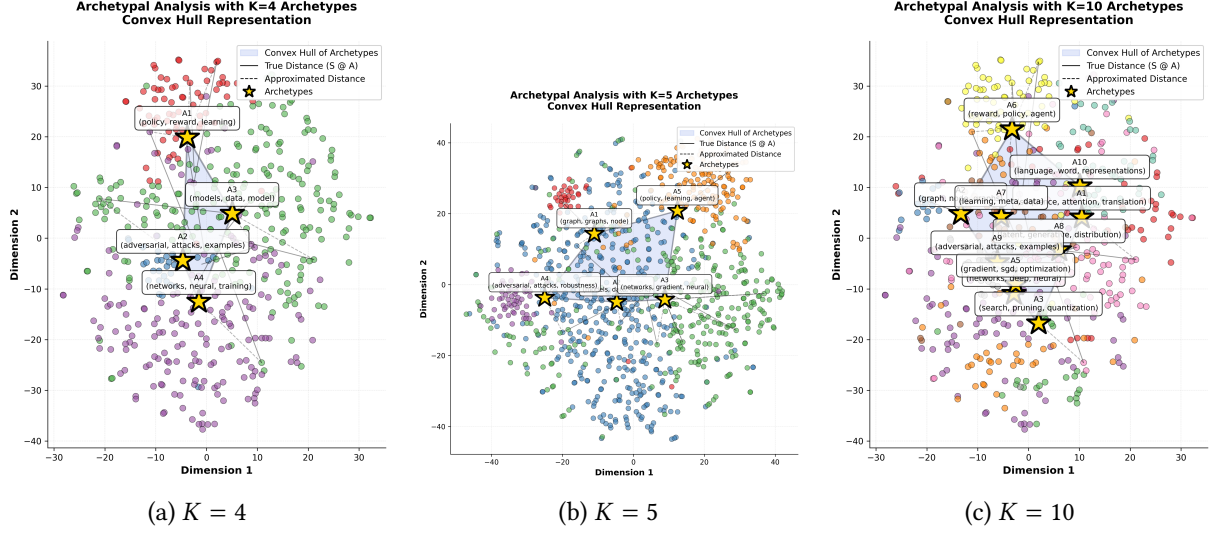


Figure 2: Two-dimensional visualization of Archetypal Analysis for ICML 2019 conference abstracts with varying numbers of archetypes ($K = 4, 5, 10$). The presence of well-defined convex hulls indicates stable and interpretable archetypal structures in the learned representations.

Comparative Visualization Across Datasets To provide a consolidated view of model behavior, Figure 5 summarizes the results obtained on the Bag-of-Words (NIPS-like) dataset, while Figure 6 presents the corresponding visualization for the ICML 2019 conference dataset. These visual summaries integrate both numerical metrics and structural patterns, enabling direct comparison across models and datasets.

A similar pattern is observed for the ICML 2019 dataset, as shown in Figure 6. Both AA and NMF exhibit moderate sparsity, with non-zero component usage of 46.6% and 51.7%, respectively. This behavior is clearly reflected in the heatmap structure, where semantic weight is concentrated in a limited number of components per document.

The results reported here correspond to the $K = 10$ setting; however, analogous experiments were conducted for $K = 4, 5$, and 10 across both datasets. The qualitative trends observed in the visualizations remain consistent across these configurations. For completeness and reproducibility, the full set of experimental results and visual summaries is made available in the accompanying GitHub repository.

Top 10 Words per Component (Grayscale = Relative Strength)										
	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
NMF	network	unit	system	function	model	learning	neuron	cell	training	image
	neural	input	control	point	data	algorithm	circuit	input	set	object
	pattern	network	neural	set	parameter	weight	input	pattern	data	feature
	architecture	hidden	dynamic	approximation	distribution	error	weight	neuron	input	map
	connection	output	controller	problem	prediction	problem	neural	field	classifier	field
	delay	weight	learning	error	term	vector	chip	visual	output	images
	recurrent	layer	signal	number	level	training	output	response	speech	features
	form	training	movement	result	probability	gradient	analog	cortex	performance	point
	result	pattern	robot	algorithm	likelihood	method	current	synaptic	recognition	visual
	nodes	representation	action	linear	structure	pattern	synapse	receptive	error	recognition
AA	cell	classifier	learning	model	function	image	unit	network	neuron	weight
	model	training	system	data	network	object	network	neural	network	network
	input	pattern	algorithm	network	input	motion	hidden	input	neural	input
	response	set	network	neural	approximation	point	input	output	input	error
	field	classification	control	function	threshold	map	learning	training	circuit	output
	cortex	error	function	learning	learning	images	output	system	model	learning
	neuron	data	model	parameter	set	system	weight	recognition	function	neural
	frequency	problem	reinforcement	set	problem	model	layer	signal	output	training
	visual	algorithm	action	training	point	field	training	set	synaptic	vector
	cortical	number	task	error	result	visual	pattern	learning	system	algorithm
K-means	neuron	network	network	cell	network	learning	network	function	model	weight
	network	unit	classifier	model	system	algorithm	neural	network	network	network
	model	input	training	input	object	function	system	input	data	learning
	neural	training	speech	neuron	unit	network	input	set	function	input
	input	output	recognition	field	image	model	neuron	learning	neural	error
	system	hidden	set	synaptic	point	system	learning	neural	system	training
	cell	learning	output	response	neural	control	chip	error	learning	algorithm
	connection	set	pattern	system	input	action	weight	unit	set	output
	function	weight	input	output	set	task	function	weight	problem	function
	noise	pattern	vector	function	field	error	output	approximation	control	unit
LDA	song	dfs	bat	art2	viip	forgeries	dolphin	parsec	network	sing
	anna	ott	erp	bfg	lid	nmc2	router	eepron	input	primed
	bird	aining	echoes	itm	harmonet	jhu	echoes	apg	learning	memorized
	peg	oed	echo	ill	ppl	cm2	echo	janus	model	hypersnet
	drt	ven	glint	analyzes	lpl	business	gateway	kata	neural	lan
	lid	alon	simmon	iti	chorale	reg	rmin	cash	function	confirming
	rapture	tie	epesicus	offline	chorales	ickinger	roittlat	spotter	unit	wtz
	rap	ste	ferragamo	multistep	chord	forger	nachtigall	jfet	system	chian
	clones	sour	replicas	hull	harmonization	genuine	penner	alphanumeric	weight	shi
	ildn	wha	dissociation	novelty	harmonies	preferences	tube	ogi	set	retrieving

Figure 3: Top words for each component ($K = 10$) extracted from the Bag-of-Words corpus using NMF, AA, k -means, and LDA. The average cosine angles between components are 81.0° , 64.9° , 51.1° , and 18.9° , respectively. These results indicate that AA extracts more geometrically distinct semantic directions than k -means and LDA. Furthermore, the proportion of non-zero entries in the document–component matrix S is 66.5% for NMF, 43.5% for AA, 10.0% for k -means, and 100% for LDA, highlighting the strong sparsity of AA relative to NMF and LDA.



Figure 4: Top words for each component ($K = 10$) extracted from the ICML 2019 conference abstracts using NMF, AA, k -means, and LDA. The average number of active components per document is 5.17 for NMF, 4.66 for AA, 1.00 for k -means, and 10.00 for LDA, indicating that AA achieves a favorable balance between sparsity and expressiveness. The average cosine angles are 79.1° , 73.8° , 64.1° , and 46.4° , respectively. AA also achieves the lowest Frobenius reconstruction error among the compared methods (21.20), demonstrating its ability to capture salient semantic structure while maintaining compact representations.

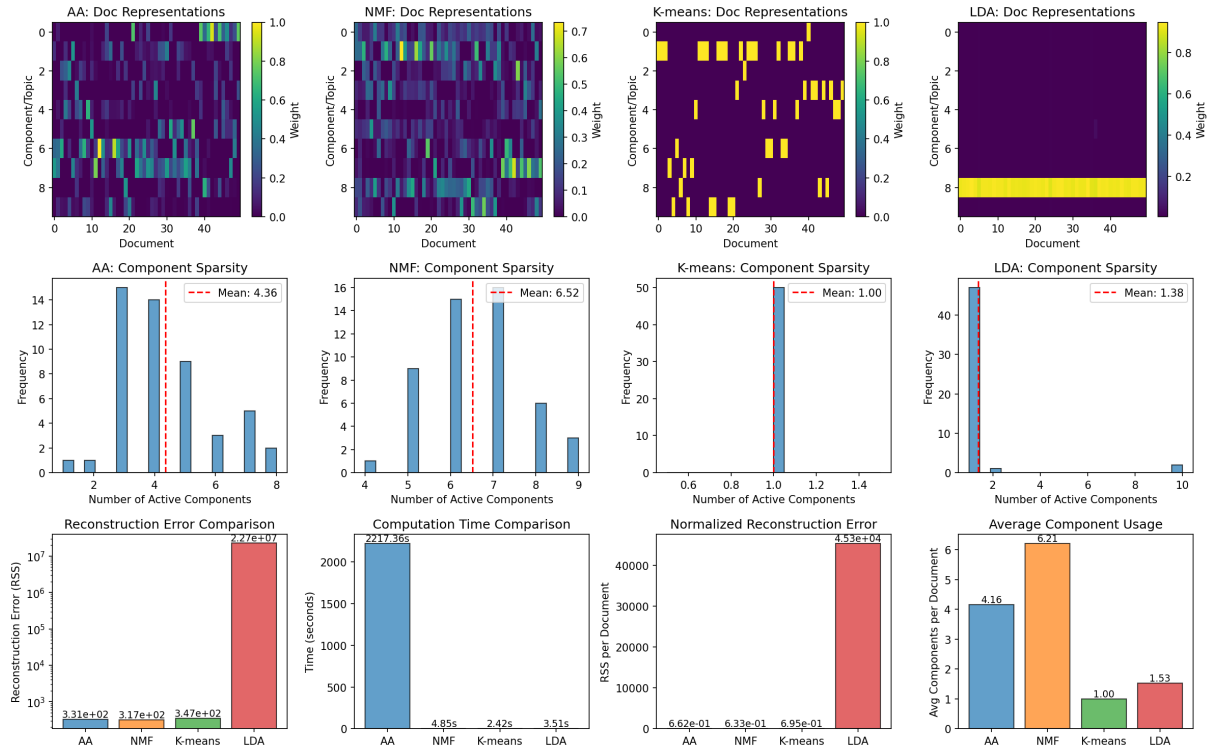


Figure 5: Summary visualization of topic modeling results on the Bag-of-Words dataset using Archetypal Analysis (AA), NMF, k -means, and LDA ($K = 10$). The heatmaps indicate that AA and NMF activate components more broadly across documents compared to k -means and LDA. These visual patterns are consistent with the quantitative metrics, particularly sparsity, cosine-angle separation, and reconstruction error.

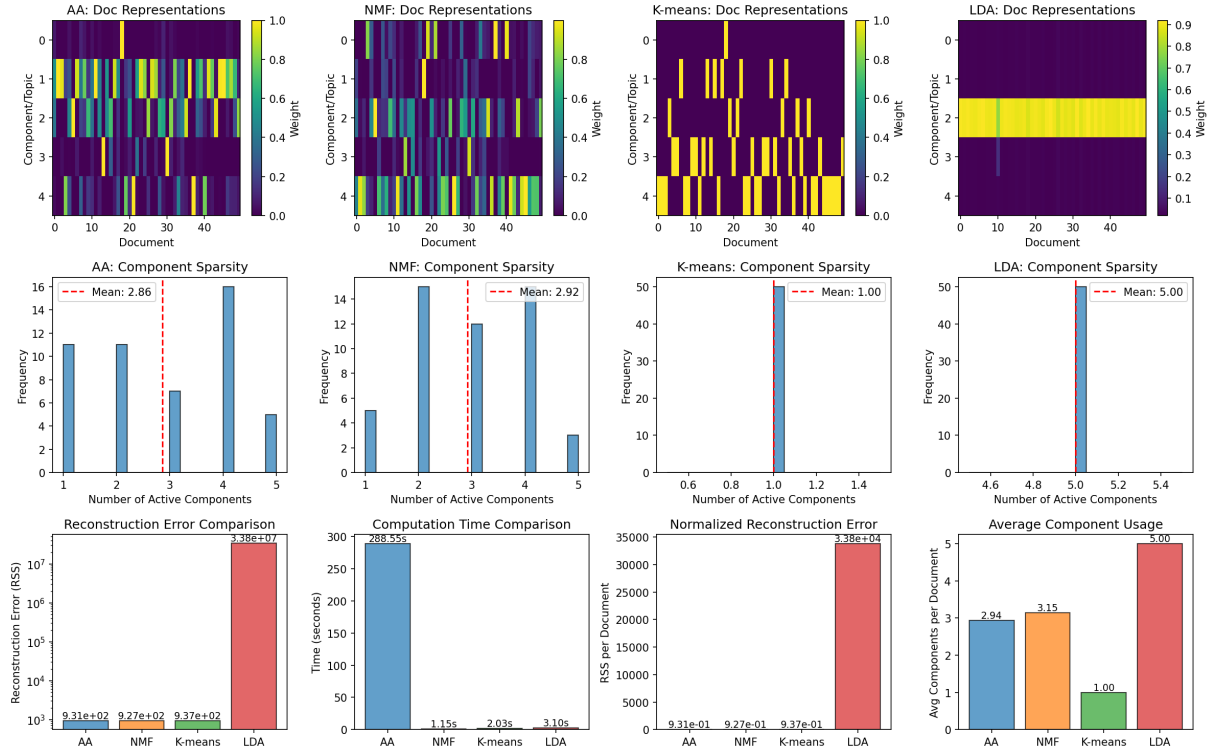


Figure 6: Summary visualization for the ICML 2019 conference dataset ($K = 10$). Archetypal Analysis and NMF demonstrate comparatively strong sparsity and structured component activation, as reflected in the heatmaps. The reconstruction errors (RSS) are 4.50×10^2 , 4.46×10^2 , 4.57×10^2 , and 2.62×10^6 for AA, NMF, k -means, and LDA, respectively. While AA incurs a higher computational cost (524.52 s), it achieves a competitive balance between reconstruction accuracy, sparsity, and geometric interpretability.